TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

TRABAJO PRÁCTICO SEMANA 3

Automatización de Calidad y Blindaje

Tecnicatura Universitaria en Programación a Distancia – UTN

Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26

Sigo con el proyecto Tienda de las semanas anteriores. Herramientas usadas: **black** (formatter), **flake8** (linter, que incluye la complejidad ciclomática), **pytest + pytest-cov** (tests y cobertura), **radon** (métricas de complejidad), **jscpd** (código duplicado) y **pip-audit** (vulnerabilidades en dependencias).

1. Pre-commit hook

El script está en .githooks/pre-commit y se activa una sola vez con git config core.hooksPath .githooks. Así queda versionado en el repositorio y todo el equipo usa el mismo.

.githooks/pre-commit

```
#!/bin/sh
# Pre-commit hook: corre en orden Formatter -> Linter -> Tests.
# Si un paso falla, el commit se cancela y los siguientes no se ejecutan.
# Instalación (una vez): git config core.hooksPath .githooks

echo "== [1/3] Formatter (black) =="
black --check src tests || {
  echo "X Hay archivos sin formatear. Corré 'black src tests' y volvé a hacer el commit."
  exit 1
}

echo "== [2/3] Linter (flake8) =="
flake8 src tests || {
  echo "X El linter encontró errores. Corregilos antes de hacer el commit."
  exit 1
}

echo "== [3/3] Tests unitarios + cobertura (pytest) =="
python3 -m pytest -q --cov --cov-report=term-missing || {
  echo "X Fallaron los tests o la cobertura está por debajo del 80%."
  exit 1
}

echo "OK: todos los controles pasaron. Commit permitido."
```

Los controles corren **en orden** y cada uno termina con exit 1 si falla. Cuando un hook termina con un código distinto de 0, git **cancela el commit**: no se guarda nada y los cambios quedan en el área de preparación (staging) para corregirlos. Además, como el script sale apenas falla un paso, los siguientes ni se ejecutan. Lo probé con tres intentos reales:

Intento 1: código sin formatear → falla el formatter

```
$ git commit -m "Agrega descuentos y cobro con pasarela (#1)"
== [1/3] Formatter (black) ==
would reformat src/tienda/precios.py

Oh no! 💥 💔 💥
1 file would be reformatted, 4 files would be left unchanged.
```




```
X Hay archivos sin formatear. Corré 'black src tests' y volvé a hacer el commit.
[código de salida: 1]
```

Black detecta un archivo mal formateado, el script muestra qué hacer y corta. El linter y los tests no se llegan a ejecutar.

Intento 2: formato corregido, pero con un bug → fallan los tests

```
$ black src tests
reformatted tienda/src/tienda/precios.py

All done! ✨ 🍰 ✨
1 file reformatted, 4 files left unchanged.

$ git add . && git commit -m "Agrega descuentos y cobro con pasarela (#1)"
== [1/3] Formatter (black) ==
All done! ✨ 🍰 ✨
5 files would be left unchanged.
== [2/3] Linter (flake8) ==
== [3/3] Tests unitarios + cobertura (pytest) ==
..............F                                                      [100%]
================================== FAILURES ===================================
_____________ test_socio_con_cupon_valido_se_queda_con_el_mayor _______________

    def test_socio_con_cupon_valido_se_queda_con_el_mayor():
        # Arrange
        total = 20000

        # Act
        resultado = aplicar_descuento(total, es_socio=True, cupon="PROMO15")

        # Assert: 15% y no 10% + 15%
>       assert resultado == 17000
E       assert 15000.0 == 17000

tests/test_precios.py:118: AssertionError
================================ tests coverage ================================
_____________ coverage: platform linux, python 3.13.17-final-0 _____________

Name                     Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------
src/tienda/__init__.py       0      0      0      0   100%
src/tienda/pagos.py         14      0      4      0   100%
src/tienda/precios.py       20      0     10      0   100%
--------------------------------------------------------------------
TOTAL                       34      0     14      0   100%
Required test coverage of 80.0% reached. Total coverage: 100.00%
============================ short test summary info ============================
FAILED tests/test_precios.py::test_socio_con_cupon_valido_se_queda_con_el_mayor
1 failed, 14 passed in 0.08s
X Fallaron los tests o la cobertura está por debajo del 80%.
[código de salida: 1]
```



Pasan el formato y el linter, pero un test detecta que los descuentos se acumulaban (daba 15000 en lugar de 17000). La consola muestra el test que falló, la línea y el valor obtenido frente al esperado. El commit se rechaza.

Intento 3: bug corregido → commit permitido

```
$ git add . && git commit -m "Agrega descuentos y cobro con pasarela (Closes #1)"
== [1/3] Formatter (black) ==
All done! ✨ 🍰 ✨
5 files would be left unchanged.
== [2/3] Linter (flake8) ==
== [3/3] Tests unitarios + cobertura (pytest) ==
.............                                                            [100%]
================================ tests coverage ================================
_____________ coverage: platform linux, python 3.13.17-final-0 _____________

Name                     Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------
src/tienda/__init__.py       0      0      0      0   100%
src/tienda/pagos.py         14      0      4      0   100%
src/tienda/precios.py       20      0     10      0   100%
--------------------------------------------------------------------
TOTAL                       34      0     14      0   100%
Required test coverage of 80.0% reached. Total coverage: 100.00%
15 passed in 0.05s
OK: todos los controles pasaron. Commit permitido.
[feature/1-descuentos-y-pagos abbe1ca] Agrega descuentos y cobro con pasarela (Closes #1)
 5 files changed, 255 insertions(+)
 create mode 100644 src/tienda/__init__.py
 create mode 100644 src/tienda/pagos.py
 create mode 100644 src/tienda/precios.py
 create mode 100644 tests/test_pagos.py
 create mode 100644 tests/test_precios.py
[código de salida: 0]
```

¿Por qué esto protege la rama principal?

Porque el código que no pasa los controles ni siquiera llega a ser un commit, entonces tampoco puede llegar a un Pull Request ni a main. Los errores se encuentran en la computadora del desarrollador, que es donde es más barato arreglarlos, y no después de integrar, cuando ya afectan a todo el equipo. Hay que tener en cuenta que el hook es local y se puede saltear con git commit --no-verify, así que lo ideal es que los mismos controles también corran en el servidor (por ejemplo con GitHub Actions) como requisito para aprobar el PR.

2. Auditoría estática

Para la auditoría usé un módulo heredado de reportes de ventas (legacy/reportes_legacy.py) con las dependencias viejas que tenía el proyecto (legacy/requirements-legacy.txt).

legacy/reportes_legacy.py

```
# Código heredado de baja calidad: se usa para la auditoría estática (TP3, punto 2).
# NO forma parte del código que se integra; está excluido de black y flake8.
import os, sys, json
```

```
def generar_reporte(ventas, tipo, moneda, incluir_iva, cliente_tipo, formato):
    total = 0
    lineas = []
    for v in ventas:
        if tipo == "diario":
            if v["estado"] == "pagada":
                if moneda == "ARS":
                    if incluir_iva:
                        monto = v["monto"] * 1.21
                    else:
                        monto = v["monto"]
                elif moneda == "USD":
                    if incluir_iva:
                        monto = v["monto"] * 1.21 / 1000
                    else:
                        monto = v["monto"] / 1000
                else:
                    monto = 0
                if cliente_tipo == "mayorista":
                    monto = monto * 0.9
                elif cliente_tipo == "minorista":
                    monto = monto
                total = total + monto
                lineas.append(str(v["id"]) + " - " + str(monto))
        elif tipo == "mensual":
            if v["estado"] == "pagada":
                if moneda == "ARS":
                    if incluir_iva:
                        monto = v["monto"] * 1.21
                    else:
                        monto = v["monto"]
                elif moneda == "USD":
                    if incluir_iva:
                        monto = v["monto"] * 1.21 / 1000
                    else:
                        monto = v["monto"] / 1000
                else:
                    monto = 0
                if cliente_tipo == "mayorista":
                    monto = monto * 0.9
                elif cliente_tipo == "minorista":
                    monto = monto
                total = total + monto
                lineas.append(str(v["id"]) + " - " + str(monto))
    if formato == "texto":
        return "\n".join(lineas) + "\nTOTAL: " + str(total)
    elif formato == "json":
        return {"lineas": lineas, "total": total}
    else:
        return None
```

legacy/requirements-legacy.txt

```
requests==2.19.1
urllib3==1.23
jinja2==2.10
pyyaml==5.3
```



Resultado del análisis

```
$ radon cc legacy/reportes_legacy.py -s
legacy/reportes_legacy.py
    F 6:0 generar_reporte - C (20)

$ flake8 --max-complexity 10 legacy/
legacy/reportes_legacy.py:3:1: F401 'os' imported but unused
legacy/reportes_legacy.py:3:1: F401 'sys' imported but unused
legacy/reportes_legacy.py:3:1: F401 'json' imported but unused
legacy/reportes_legacy.py:3:10: E401 multiple imports on one line
legacy/reportes_legacy.py:6:1: C901 'generar_reporte' is too complex (20)

$ npx jscpd --min-lines 5 --format python legacy/
Clone found (python):
 - legacy/reportes_legacy.py [30:31 - 50:7] (20 lines, 197 tokens)
   legacy/reportes_legacy.py [10:28 - 30:13]

Resumen: 1 archivo, 55 líneas, 1 clon, 20 líneas duplicadas (36.36%)
Found 1 clones.

$ pip-audit -r legacy/requirements-legacy.txt
Name     Version ID              Fix Versions
-------- ------- --------------- -------------
requests 2.19.1  PYSEC-2018-28   2.20.0
requests 2.19.1  PYSEC-2018-28   2.20.0
requests 2.19.1  PYSEC-2023-74   2.31.0
requests 2.19.1  PYSEC-2023-74   2.31.0
requests 2.19.1  PYSEC-2026-1873 2.32.0
requests 2.19.1  PYSEC-2026-1872 2.32.4
requests 2.19.1  PYSEC-2026-2275 2.33.0
requests 2.19.1  PYSEC-2026-1873 2.32.0
requests 2.19.1  PYSEC-2026-1872 2.32.4
requests 2.19.1  PYSEC-2026-2275 2.33.0
... (se muestran las primeras filas)
Found 49 known vulnerabilities in 4 packages
```

Puntos críticos encontrados

Problema	Métrica	Explicación
Complejidad ciclomática alta	radon: 20 (rango C) flake8 C901	La función tiene 20 caminos posibles, el doble del máximo de la DoD (10). Para probarla bien harían falta al menos 20 tests, y es muy fácil romper algo al tocarla.
Código duplicado	jscpd: 20 líneas (36%)	El bloque de "diario" y el de "mensual" son idénticos. Si cambia el IVA hay que acordarse de modificarlo en dos lugares.
Imports sin usar	flake8: F401 y E401	os, sys y json no se usan. Ensucian el código y confunden al que lo lee.
Dependencias vulnerables	pip-audit: 49 hallazgos en 4 paquetes	requests 2.19.1, urllib3 1.23, jinja2 2.10 y pyyaml 5.3 tienen vulnerabilidades conocidas con versión corregida disponible.



Estrategia de refactorización

- **1. Primero, red de seguridad:** antes de cambiar nada escribí tests que comparan el resultado del código nuevo con el del viejo para todas las combinaciones de parámetros (162 casos). Así me aseguro de que la refactorización no cambia el comportamiento.
- **2. Eliminar la duplicación:** los bloques "diario" y "mensual" hacían lo mismo, así que quedó uno solo.
- **3. Extraer funciones:** separé la lógica en funciones chicas con una sola responsabilidad: convertir_moneda, calcular_monto y _formatear.
- **4. Reducir anidamiento:** usé retornos tempranos y continue en lugar de if dentro de if, y saqué los "números mágicos" (1.21, 1000, 0.9) a constantes con nombre.
- **5. Actualizar dependencias:** pasé cada paquete a una versión sin vulnerabilidades conocidas y lo verifiqué con pip-audit. Para que no vuelva a pasar, pip-audit debería correr en el pipeline de CI.

src/tienda/reportes.py (refactorizado)

```
"""Reporte de ventas (versión refactorizada de legacy/reportes_legacy.py)."""

IVA = 1.21
COTIZACION_USD = 1000
DESCUENTO_MAYORISTA = 0.9


def convertir_moneda(monto, moneda, incluir_iva):
    """Aplica el IVA si corresponde y convierte a la moneda pedida."""
    if incluir_iva:
        monto = monto * IVA
    if moneda == "ARS":
        return monto
    if moneda == "USD":
        return monto / COTIZACION_USD
    return 0


def calcular_monto(venta, moneda, incluir_iva, cliente_tipo):
    """Monto final de una venta: moneda, IVA y descuento por tipo de cliente."""
    monto = convertir_moneda(venta["monto"], moneda, incluir_iva)
    if cliente_tipo == "mayorista":
        monto = monto * DESCUENTO_MAYORISTA
    return monto


def generar_reporte(ventas, tipo, moneda, incluir_iva, cliente_tipo, formato):
    """Genera el reporte de ventas pagadas en formato texto o json."""
    if tipo not in ("diario", "mensual"):
        return _formatear([], 0, formato)

    lineas = []
    total = 0
    for venta in ventas:
        if venta["estado"] != "pagada":
            continue
        monto = calcular_monto(venta, moneda, incluir_iva, cliente_tipo)
        total += monto
```




```
        lineas.append(f"{venta['id']} - {monto}")
    return _formatear(lineas, total, formato)


def _formatear(lineas, total, formato):
    if formato == "texto":
        return "\n".join(lineas) + f"\nTOTAL: {total}"
    if formato == "json":
        return {"lineas": lineas, "total": total}
    return None
```

requirements.txt (actualizado)

```
# Dependencias actualizadas (antes en legacy/requirements-legacy.txt)
requests==2.34.2
urllib3==2.8.0
jinja2==3.1.6
pyyaml==6.0.3
```

Resultado después de refactorizar

```
$ radon cc src/tienda/reportes.py -s
src/tienda/reportes.py
    F 8:0 convertir_moneda - A (4)
    F 27:0 generar_reporte - A (4)
    F 43:0 _formatear - A (3)
    F 19:0 calcular_monto - A (2)

$ flake8 --max-complexity 10 src/
(sin errores)

$ npx jscpd --min-lines 5 --format python src/
Found 0 clones.

$ pip-audit -r requirements.txt
No known vulnerabilities found
```

Métrica	Antes	Después
Complejidad ciclomática máxima	20 (C)	4 (A)
Código duplicado	20 líneas (36%)	0
Errores de flake8	5	0
Vulnerabilidades en dependencias	49 hallazgos	0
Tests del módulo	ninguno	163, todos pasan

3. Informe de Calidad en Verde

a) Trazabilidad Issue → Pull Request

Cada cambio sale de un issue, se hace en su propia rama y el commit lo cierra con "Closes #N". Al subir el repositorio a GitHub, esa palabra clave enlaza el PR con el issue y lo cierra automáticamente al integrarlo.

Issue	Rama	Commit	Pull Request (merge)
#1 feature: descuentos y cobro con pasarela	feature/1-descuentos-y-pagos	abbe1ca "Closes #1"	PR #3 → d8c096f
#2 technical-debt: refactor de reportes y dependencias	tech-debt/2-refactor-reportes	c2b9990 "Closes #2"	PR #4 → 9073c01

```
$ git log --graph --oneline --all
*   9073c01 Merge PR #4: Refactor de reportes y dependencias (Closes #2)
|\
| * c2b9990 Refactoriza reportes y actualiza dependencias (Closes #2)
|/
*   d8c096f Merge PR #3: Descuentos y cobro con pasarela (Closes #1)
|\
| * abbe1ca Agrega descuentos y cobro con pasarela (Closes #1)
|/
* a690217 Gobernanza: DoD, plantillas de issue y PR, etiquetas y pre-commit hook
```

Nota: el repositorio lo trabajé de forma local, así que los merges representan a los PR. Al publicarlo en GitHub, los enlaces quedan como github.com/<usuario>/tienda/issues/1 y github.com/<usuario>/tienda/pull/3.

b) Logs del pre-commit hook

Los tres intentos del punto 1 son la evidencia: dos commits rechazados (por formato y por un test que falla) y uno aceptado. El commit del issue #2 también pasó los tres controles (formatter, linter y 178 tests).

c) Reporte de cobertura

Coverage report: 100%

Files | Functions | Classes

coverage.py v7.17.0, created at 2026-11-02 20:45 +0000

	Statements				Branches			
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	c
src/tienda/__init__.py	100%	0	0	0	100%	0	0	
src/tienda/pagos.py	100%	14	0	0	100%	4	0	
src/tienda/precios.py	100%	20	0	0	100%	10	0	
src/tienda/reportes.py	100%	34	0	0	100%	18	0	
Total	**100%**	**68**	**0**	**0**	**100%**	**32**	**0**	

```
$ pytest --cov --cov-report=term-missing --cov-report=html
_____________ coverage: platform linux, python 3.13.17-final-0 _____________

Name                      Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------
src/tienda/__init__.py        0      0      0      0   100%
src/tienda/pagos.py          14      0      4      0   100%
src/tienda/precios.py        20      0     10      0   100%
src/tienda/reportes.py       34      0     18      0   100%
---------------------------------------------------------------------
TOTAL                        68      0     32      0   100%
```




```
Coverage HTML written to dir htmlcov
Required test coverage of 80.0% reached. Total coverage: 100.00%
178 passed in 0.43s
```

La cobertura de sentencias y la de ramas son del **100%**, por encima del umbral del 80% de la DoD. Los 178 tests pasan.

d) Diagnóstico final de análisis estático

Sobre el código que se integra (src/): flake8 sin errores, complejidad máxima de 4 (rango A, el límite es 10), sin código duplicado y pip-audit sin vulnerabilidades conocidas.

Verificación de la Definition of Done

Criterio	Estado	Evidencia
1. Integridad de compilación	OK	El paquete se importa y los 178 tests se ejecutan sin errores.
2. Pruebas automatizadas	OK	178 tests, 0 fallos (logs del hook).
3. Cobertura mínima (80%)	OK	100% de ramas (reporte de cobertura).
4. Revisión por pares	Pendiente	Se cumple cuando otro integrante aprueba el PR en GitHub.
5. Gobernanza de estilo	OK	black y flake8 sin errores; complejidad máxima 4.
6. Documentación técnica	OK	Docstrings en las funciones públicas y commits con "Closes #N".

Dictamen

El desarrollo cumple los controles automáticos de la DoD: los tests pasan, la cobertura de ramas está en 100%, el código pasa el formatter y el linter, y no hay vulnerabilidades en las dependencias. Por lo tanto, se considera **apto para integrarse a main**, sujeto a la aprobación del Pull Request por otro integrante del equipo (criterio 4).